Assignment 5 — End-to-End SQL Data Engineering Mini Project

Database Design • Advanced Querying • Programmability • Transactions • Incremental Loading • Analytics • Performance Tuning

Assignment Goal

Demonstrate comprehensive mastery of SQL Data Engineering. You will design and create database objects using DDL, write analytical SQL using scalar and aggregate functions, joins, set operations, subqueries, and CTEs, create reusable SQL objects using Views, Stored Procedures, and Functions, enforce data consistency using Transactions and Triggers, perform incremental synchronization using MERGE, apply Window Functions for advanced analytics, and optimize queries through Indexing, Sargability, Execution Plans, and Statistics.

Learning Outcomes

- Implement DDL objects using appropriate data types, keys, constraints, and defaults.
- Write SQL queries using filtering, sorting, CASE expressions, scalar functions, and NULL handling.
- Use joins and set operations to combine and compare relational datasets.
- Build reusable queries using subqueries and CTEs.
- Work with local temporary tables and table variables.
- Create reusable SQL objects using Views, Stored Procedures, and User-Defined Functions.
- Guarantee data consistency using Transactions, TRY/CATCH, and ROLLBACK.
- Automate auditing using Triggers.
- Implement incremental synchronization using MERGE and staging tables.
- Handle missing records, safe reruns, and idempotent processing.
- Apply Window Functions for ranking, latest-record selection, trend analysis, and cumulative calculations.
- Read execution plans and improve query performance using indexes, Sargability, and Statistics.

Database Setup Instructions (MANDATORY)

Required Database

For this assignment, you will use the official Microsoft "Northwind" Sample Database. Do NOT create your own tables or assume data types.
Database Name: Northwind
Database Type: Relational SQL Server Database

https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs

Installation Steps

1. Open the link above and locate `instnwnd.sql` in the repository.
2. Download or save the `instnwnd.sql` script to your computer.
3. Open SQL Server Management Studio (SSMS) or Azure Data Studio.
4. Open the downloaded `instnwnd.sql` file in a new query window and click Execute (or Run). This will create and populate the Northwind database.
5. Begin your assignment script with `USE Northwind; GO`.

General Instructions

• Write your queries using standard SQL Server syntax.
• Use the exact column and table names as defined in the Northwind database.
• Submit a single `.sql` file containing all tasks, clearly separated by comments.
• Use comments such as `-- Task 1.1`, `-- Task 1.2`, `-- Task 2.1`.
• Your script should execute successfully from top to bottom.
• Create required DDL objects before executing or testing dependent objects.
• Do not hard-code query results instead of writing SQL logic.
• Do not use external datasets.
• Do not redesign the Northwind database or submit ERD diagrams for this assignment.
• Your SQL file should be readable and professionally structured.

Task 1 — DDL & Database Design

Weight: **10 marks**

1.1 Project Audit Log

Write the DDL statement to CREATE a new table named `Project_AuditLog`.
The table must contain:
• `LogID`: An integer that auto-increments (Primary Key).
• `TableName`: A string of max 50 characters (Not Null).
• `ActionPerformed`: A string of max 50 characters (Not Null).
• `ActionDate`: A datetime column that defaults to the current system date and time.

1.2 ALTER TABLE

Use an `ALTER TABLE` statement in your script to modify a project-created table.
The modification must demonstrate a valid schema-level change such as adding a constraint or another appropriate column definition.
Do not modify the existing Northwind business logic unnecessarily.

1.3 DDL Awareness

At the end of Task 1, include a SQL comment identifying where `CREATE TABLE`, `ALTER TABLE`, and `DROP TABLE` belong in the database-object lifecycle.
You do not need to redesign an ERD for this assignment.

Task 2 — SQL Fundamentals & Scalar Functions

Weight: **10 marks**

2.1 Product Formatting and Categorization

Write a `SELECT` query on the `[Products]` table.
Display:
• `ProductName` in entirely UPPERCASE.
• A new column named `PriceCategory`.
Use a `CASE` expression:
• If `UnitPrice > 50`, label it `Premium`.
• Otherwise, label it `Standard`.

2.2 Filtering, Sorting, and Data-Type Conversion

Write a query on `[Products]` that:
• Filters products based on a meaningful business condition.
• Sorts the result by `UnitPrice` in descending order.
• Uses `CAST` or `CONVERT` to display `UnitPrice` in an explicitly chosen numeric format.

2.3 NULL Handling

Write a query that demonstrates how NULL values can be handled safely in a Northwind column that allows NULLs.
The query must clearly distinguish between:
• A real value.
• A missing / NULL value.
Use an appropriate SQL expression such as `ISNULL`, `COALESCE`, or `CASE`.

Task 3 — Joins, Set Operations, & Aggregations

Weight: **10 marks**

3.1 Set Operations

Write a query using `UNION` to combine the `City` and `Country` columns from the `[Customers]` table with the `City` and `Country` columns from the `[Employees]` table.
Ensure no duplicates.

3.2 Aggregations

INNER JOIN `[Categories]` and `[Products]`.
Use `GROUP BY` to display:
• `CategoryName`
• The COUNT of products in each category.
Use `HAVING` to show only categories with more than 10 products.

3.3 SELF JOIN

Use a `SELF JOIN` on `[Employees]` to display:
• Employee name.
• Manager name.
Use the `ReportsTo` relationship.
Your query must correctly handle employees who do not have a manager.

3.4 INTERSECT

Write a set-operation query using `INTERSECT` to identify cities that appear in both `[Customers]` and `[Employees]`.

Task 4 — Subqueries, CTEs, & Intermediate Query Techniques

Weight: **10 marks**

4.1 CTE

Create a CTE named `EmpSales` that calculates the Total Sales:
• `SUM(Quantity * UnitPrice)`
per `EmployeeID` from the `[Order Details]` and `[Orders]` tables.
Join this CTE with the `[Employees]` table to display:
• Employee's `FirstName`
• Total Sales

4.2 Anti-Join / Subquery

Write a query using a `NOT EXISTS` subquery or `LEFT JOIN IS NULL` to find all `CustomerID`s in `[Customers]` that have NEVER placed an order in `[Orders]`.

4.3 Correlated Subquery

Write a correlated subquery that finds products whose `UnitPrice` is greater than the average `UnitPrice` of products in the same category.
The inner query must depend on the current row of the outer query.

4.4 Local Temporary Table

Create a local temporary table named `#TopProducts`.
Use it to store a suitable filtered subset of products and then query the temporary table.

4.5 Table Variable

Declare a table variable named `@TopCategories`.
Insert appropriate category information into it and query the variable.

4.6 Global Temporary Table — Conceptual Checkpoint

Do NOT create a global temporary table in the submitted execution script.
Instead, add a SQL comment explaining:
• How `##Temp` differs in scope from `#Temp`.
• Why a global temporary table should be used carefully in a shared student SQL Server environment.

Task 5 — Views, Procedures, & Functions

Weight: **10 marks**

5.1 Standard View

Create a view named:
`vw_CustomerOrderSummary`
The view must provide a useful summary containing appropriate customer/order information, such as:
• `CustomerID`
• `CompanyName`
• Number of Orders
Use an appropriate join and aggregation.

5.2 Stored Procedure

Write the DDL to CREATE a Parameterized Stored Procedure named:

`sp_GetCustomerOrders`
It must:
• Accept `@CustomerID` as `NCHAR(5)`.
• Return all columns from the `[Orders]` table for that specific customer.
Include an `EXEC` statement to test it with:

```sql
CustomerID = 'ALFKI'
```

5.3 User-Defined Function

Create a scalar User-Defined Function named:
`fn_GetCustomerOrderCount`
It must accept:

```sql
@CustomerID NCHAR(5)
```

and return the number of orders associated with that customer.
Test the function using at least one valid Northwind `CustomerID`.

5.4 Indexed / Materialized View Awareness

Add a short SQL comment explaining:
• How a normal logical view differs conceptually from an indexed/materialized view.
• Why an indexed/materialized view may be useful for frequently reused expensive summaries.
• Why maintenance or refresh strategy matters.
You do not need to implement a production indexed/materialized view for this assignment.

Task 6 — Transactions & Error Handling

Weight: **10 marks**

6.1 Explicit Transaction

Write a `TRY/CATCH` block containing an explicit Transaction.
Inside the `TRY` block:
• BEGIN the transaction.
• UPDATE the `UnitPrice` of `ProductID = 1` by adding `$5`.
• Intentionally cause a divide-by-zero error using:

```sql
SELECT 1/0;
```

6.2 Error Handling

Inside the `CATCH` block:
• ROLLBACK the transaction.
• Use `SELECT ERROR_MESSAGE()` to return the error details.
The final database state must not contain the price change because of the rollback.

6.3 Savepoint

Before the update, create a named transaction savepoint.
Demonstrate that you understand the role of:

```sql
SAVE TRANSACTION
```

Your final error-handling path must still guarantee that the failed transaction does not leave unintended changes.

6.4 Transaction Concepts

Add SQL comments briefly explaining:
• ACID.
• Blocking.
• Deadlock.
Do not intentionally create a deadlock in the submitted script.

Task 7 — Triggers for Automation

Weight: **10 marks**

7.1 UPDATE Trigger

Write the DDL to CREATE an `AFTER UPDATE` trigger named:
`trg_AuditProducts`
on the `[Products]` table.
Whenever a product is updated, it must automatically INSERT a record into:
`Project_AuditLog`
with:
• `TableName = 'Products'`
• `ActionPerformed = 'Product Updated'`

7.2 DELETE Trigger

Create an `AFTER DELETE` trigger on `[Products]`.
The trigger must automatically insert an audit record into `Project_AuditLog` with:
• `TableName = 'Products'`
• `ActionPerformed = 'Product Deleted'`

7.3 Trigger Safety

Add comments explaining:
• Why triggers should be kept small and controlled.
• Why trigger logic can introduce hidden performance or maintenance costs.
• Why trigger code should be written with multi-row statements in mind.

7.4 INSTEAD OF Trigger — Conceptual Checkpoint

Add a SQL comment explaining one situation where an `INSTEAD OF` trigger could be useful, such as when controlling operations against a view.
Do not create an `INSTEAD OF` trigger for this assignment.

Task 8 — MERGE, Incremental Loading, & Delete Handling

Weight: **10 marks**

8.1 Staging Table

Create a temporary table:

`#Staging_Shippers`
with:
• `ShipperID` as `INT`
• `CompanyName` as `NVARCHAR(50)`
Insert these two records:

```
SQL
(1, 'Speedy Express V2')
(4, 'Global Logistics')
```

8.2 MERGE Synchronization

Write a `MERGE` statement synchronizing `[Shippers]` as the Target with `#Staging_Shippers` as the Source.
Match on:
`ShipperID`
Required behavior:
• `WHEN MATCHED`: UPDATE the `CompanyName`.
• `WHEN NOT MATCHED BY TARGET`: INSERT the new `ShipperID` and `CompanyName`.

8.3 Incremental Extraction / Watermark

Declare a watermark variable using an appropriate Northwind date column.
Write a query that returns only records newer than that watermark.
Demonstrate the idea of incremental extraction rather than repeatedly processing the complete dataset.

8.4 Delete Handling

Write a query that identifies target `[Shippers]` records that are missing from `#Staging_Shippers`.
Do NOT automatically delete the records.
Add SQL comments explaining:
• Soft Delete.
• Hard Delete.
• Why delete handling should be explicitly controlled.

8.5 Idempotency

Add a SQL comment explaining why rerunning the synchronization should not create duplicate records.

Task 9 — Advanced Analytics with Window Functions

Weight: **10 marks**

9.1 Ranking

Write a query to rank products within each `CategoryID` based on `UnitPrice` in descending order.
Use:
• `RANK()`
• `DENSE_RANK()`
Display:
• `CategoryID`
• `ProductName`
• `UnitPrice`
• Both ranking results.

9.2 Trend Analysis with LAG

Query the `[Orders]` table to display:
• `OrderID`
• `CustomerID`
• `OrderDate`
• The OrderDate of the PREVIOUS order placed by the same customer.
Use the `LAG()` window function.

9.3 Latest Record Selection

Use:
`ROW_NUMBER()`
partitioned by `CustomerID` and ordered by `OrderDate DESC`.
Return only the latest order for every customer.

9.4 NTILE

Use:
`NTILE(4)`
to divide products into four approximately equal price groups based on descending `UnitPrice`.
Display the assigned quartile.

9.5 Running Total

Create a query that calculates a running cumulative value using:

```sql
SUM(...) OVER(ORDER BY ...)
```

The calculation must accumulate from the beginning of the ordered result through the current row.

Task 10 — Query Optimization, Indexing, Partitioning, & Statistics

Weight: **10 marks**

10.1 Covering Index

Write the DDL to CREATE a `NONCLUSTERED INDEX` named:
`IX_Orders_CustomerID`
on the `[Orders]` table for the `CustomerID` column.
INCLUDE:
• `OrderDate`
• `Freight`
The purpose is to help prevent unnecessary Key Lookups for queries requesting these columns.

10.2 Sargability

The following query causes a non-Sargable predicate:

```sql
SELECT OrderID FROM Orders WHERE YEAR(OrderDate) = 1997;
```

Rewrite it so that the predicate is Sargable and allows the optimizer to use an appropriate index when beneficial.

10.3 Execution Plan

Use the actual execution plan in SSMS or Azure Data Studio.
Compare the access pattern of:
• The original non-Sargable query.
• The Sargable rewritten query.
Identify whether the plan contains:
• Scan
• Seek
• Lookup
IMPORTANT:
Because the Northwind database is small, the optimizer may not always choose an Index Seek even for a Sargable query.
Your grade should therefore focus on whether you correctly interpret the execution plan rather than requiring a specific operator on the small sample database.

10.4 Statistics

Execute:

```sql
UPDATE STATISTICS Orders;
```

Add a SQL comment explaining why database statistics are important to the Query Optimizer.

10.5 Index Engineering Concepts

Add short SQL comments defining:
• B-Tree.
• Selectivity.
• Clustered Index.
• Non-Clustered Index.
• Composite Index.

10.6 Partitioning — Conceptual Checkpoint

Do NOT physically partition the Northwind database.
Instead, add a SQL comment identifying:
• A suitable date column that could be used to partition a production-scale Orders/Sales table.
• Why partitioning by date can support Partition Elimination.

Marking Rubric

Area	Marks	Evaluation Focus
T1 — DDL & Database Design	10	Correct table creation, DDL usage, data types, keys, defaults, and schema awareness
T2 — Fundamentals & Scalar Functions	10	Accurate SELECT logic, CASE, filtering, sorting, conversion, and NULL handling
T3 — Joins, Sets & Aggregations	10	Correct JOIN logic, SELF JOIN, UNION, INTERSECT, GROUP BY, and HAVING
T4 — Subqueries, CTEs & Temp Structures	10	Correct CTEs, correlated/non-correlated logic, NOT EXISTS, temp tables, and table variables
T5 — Views, Procedures & Functions	10	Reusable SQL objects and appropriate encapsulation
T6 — Transactions & Error Handling	10	ACID awareness, TRY/CATCH, transactions, rollback, and savepoints
T7 — Triggers	10	Correct auditing, trigger logic, safety, and maintainability awareness
T8 — MERGE & Incremental Loading	10	Staging, matching logic, incremental extraction, delete handling, and idempotency
T9 — Window Functions	10	Ranking, LAG, ROW_NUMBER, NTILE, and running totals
T10 — Optimization	10	Covering indexes, sargability, execution plans, statistics, and indexing concepts
Total	**100**	**Professional execution of an end-to-end SQL Data Engineering mini-project**

Submission Checklist

- The official Northwind database is used.
- The script begins with `USE Northwind; GO`.
- All required project-created objects are created before dependent objects are executed.
- The `.sql` script runs from top to bottom without syntax errors.
- Required stored procedures, views, functions, triggers, tables, and indexes are created correctly.
- The transaction test leaves no unintended price modification after rollback.
- The audit triggers write to `Project_AuditLog` correctly.
- The MERGE handles both matched and new records.
- Window Functions are partitioned and ordered correctly.
- The Sargable rewrite is logically equivalent to the original query.
- Execution plans are inspected and interpreted correctly.
- Statistics are updated.
- The script is clearly separated into:
- `-- Task 1.1`
- `-- Task 1.2`
- `...`
- `-- Task 10.6`
- No screenshots are submitted in place of SQL code.
- The final submission is one professional `.sql` file.